

Glass Ballroom Verification: Multi-Surface Attestation Without Trusted Clients

Anonymous Author

This paper is under double blind review

Abstract

Web applications increasingly rely on client-side computation to produce user-visible state, but this execution occurs in environments that cannot be trusted. Servers can verify what data was delivered—not whether the rendered interface faithfully reflects application semantics. Existing approaches address this through transport integrity or trusted execution, but neither assumption holds for dynamically rendered client-visible state in open browser environments.

We present Glass Ballroom Verification (GBV), a protocol that fills this gap without demanding execution correctness—a guarantee that simply cannot be made in untrusted browser runtimes. Instead, GBV provides a weaker but deployable property: *attestation through coordination cost*. By checking semantic consistency across multiple independently derived representations, GBV treats observable interface artifacts as evidence and performs all interpretation at the verifier. A verifier-defined observation plan, combined with invariant evaluation and deterministic canonicalization, produces cryptographic commitments without trusted hardware or cooperative infrastructure.

Evaluation across two structurally distinct web applications shows that increasing surface diversity raises the cost of coherent forgery while maintaining robustness under benign variability. Attacker success drops from 0.78 at $k = 1$ to 0.02 at $k = 6$, while implementation effort grows by more than an order of magnitude. Practical assurance in adversarial client environments can be achieved by enforcing cross-surface agreement rather than execution integrity.

1 Introduction

In a glass ballroom observed from multiple angles, a performance is judged not by any single viewpoint but by whether the choreography remains consistent across independent perspectives. A movement that appears correct from one angle may reveal inconsistency from another. Coherence across views establishes validity [2]. The remainder of this paper formalizes this intuition in operational and protocol terms.

Modern web applications operate under a different verification model than traditional distributed systems. While servers transmit authenticated data, the computation that produces user-visible state occurs inside a programmable browser runtime beyond the server’s trust boundary. Scripts execute asynchronously. Frameworks transform interface structure dynamically, and presentation evolves through client-side computation. This separation is intentional: contemporary web architectures explicitly treat client execution as untrusted, enforced through browser isolation mechanisms such as the Same-Origin Policy and sandboxed execution models [3, 9, 16].

Existing verification mechanisms address transport integrity and execution within controlled environments. Transport security protocols verify delivery integrity but provide no assurance about how received data is interpreted or rendered [17]. Trusted execution and platform attestation bind guarantees to controlled runtimes or cooperative infrastructure [6, 15]. Snapshot comparison and signed artifacts assume stable rendering semantics. None of these hold when rendered state arises from ongoing runtime computation inside a user-controlled browser. The verification gap that results is structural, not incidental.

Consider a web platform displaying completion status for a workflow that gates access to restricted resources. A manipulated client can alter the visible progress indicator to show successful completion while leaving alternate navigation views, embedded metadata, and derived summaries inconsistent with that state. Because all server-side protocol exchanges remain valid, the server observes no anomaly. The inconsistency exists only across independently rendered client surfaces. Detecting it requires checking agreement across those representations rather than trusting any single output.

We call this the *glass ballroom problem*: determining whether client-visible application state remains semantically coherent across independent representations, despite execution occurring outside the verifier’s control.

Attestation Without Correctness

Before introducing GBV, a design stance deserves explicit acknowledgment: this work does not attempt to verify that the client correctly executed the application. In open browser environments, execution correctness guarantees require trusted hardware roots that are simply unavailable across much of the web. GBV treats this not as a limitation to paper over, but as a constraint to design within.

TLS proves delivery. TEEs prove execution. GBV addresses neither, filling instead the gap between them: the consistency of dynamically rendered client-visible state. Forging one view is easy. Forging multiple independent views coherently—under a nonce-bound time window, in response to unpredictable verifier challenges—is measurably harder. In this work, *attestation* denotes verifier-derived cryptographic commitments over observed client-visible state, not measurements rooted in trusted hardware.

1.1 Overview of Glass Ballroom Verification

Glass Ballroom Verification (GBV) instantiates this model by evaluating semantic consistency across multiple independent observations [2, 10], rather than trusting any single rendered view. Unlike N-version programming, GBV does not assume independently implemented replicas or fault isolation between views; it operates over multiple representations produced by a single potentially adversarial execution environment, relying on structural and semantic divergence across rendering and data-derivation paths.

The protocol involves three entities: a *verifier* operated by the relying party, an *observation client* executing in a user-controlled browser, and a *target application*. The verifier defines observation requirements and semantic invariants. The client gathers synchronized artifacts across specified surfaces and transmits them. The verifier performs semantic extraction, invariant evaluation, and canonicalization—entirely independently of the adversarial runtime.

Conceptually, the protocol proceeds as follows:

1. The verifier issues a nonce-bound temporal challenge defining required observation surfaces and a bounded validity window.
2. The observation client captures synchronized artifacts for each surface.
3. Captured artifacts are transmitted to the verifier.
4. The verifier extracts semantic facts, evaluates cross-surface invariants, and canonicalizes the resulting fact set.
5. If invariants hold, the verifier commits the canonicalized set cryptographically, producing an attestation.

GBV does not make client-side forgery impossible. It increases the engineering effort required to maintain consistent falsified state across independent representations within a constrained time window. The non-trivial challenge is defining invariants that remain stable under benign interface variability while forcing inconsistencies under adversarial manipulation—and canonicalizing dynamically computed state despite asynchronous execution and adversarially controlled timing.

GBV complements approaches that rely on trusted execution, instrumentation, or cooperative infrastructure, and is designed for settings where those assumptions do not hold.

Contributions

- Formalization of the glass ballroom problem for client-rendered state verification in adversarial browser environments.
- A verification model and protocol that eliminates reliance on trusted execution by enforcing cross-surface semantic coherence under minimal trust assumptions.
- A semantic invariant framework designed to tolerate benign interface variability while detecting cross-view inconsistency.
- A deterministic canonicalization process that produces stable cryptographic commitments despite dynamic interface behavior.
- A reference implementation and evaluation demonstrating deployability without trusted hardware, privileged APIs, or application-specific instrumentation.

2 Threat Model and Security Objectives

GBV operates in adversarial client environments in which the prover controls local execution and browser runtime behavior. The adversary may instrument or modify execution semantics, including JavaScript execution order, runtime state, network communication, and timing behavior, and may delay, replay, or selectively respond to verifier challenges.

No trusted hardware, secure enclaves, or cooperative platform APIs are assumed. The verifier observes only network-level communication and artifacts explicitly produced through the GBV protocol. Execution integrity at the client cannot be relied upon; all guarantees must arise from properties of observable state, as captured through verifier-requested observation surfaces.

GBV operates over a set of observation surfaces $\{S_1, \dots, S_k\}$ (e.g., DOM state, embedded metadata, navigation-derived views, or rendered interface artifacts). These surfaces arise from distinct rendering or data-derivation paths that must be jointly controlled by an adversary, even when originating from a common underlying execution.

For each surface S_i , the verifier receives raw artifacts and extracts a semantic fact representation F_i . These are aggregated into F and checked against a verifier-defined invariant set C . All semantic extraction and invariant evaluation occur independently of client execution.

2.1 Adversarial Capabilities

We model increasingly capable adversaries; each class subsumes the previous.

1. **Manual manipulation.** The adversary directly modifies rendered interface state using browser developer tools or equivalent mechanisms.
2. **Automated forgery.** The adversary performs scripted manipulation, including dynamic DOM rewriting, API interception, and runtime masking synchronized with verifier challenges.
3. **Simulated environments.** The adversary operates a virtualized or instrumented client capable of generating internally consistent but fraudulent observations across all required surfaces, analogous to adversaries considered in remote attestation settings [6].

Adversaries may partially cooperate with protocol execution while selectively deviating to maintain inconsistent semantic representations. The verifier randomizes challenge selection and invariant instantiation per round; adversaries observe issued challenges but do not know future selections. Maintaining cross-surface consistency under these conditions requires tightly synchronized manipulation that grows in complexity with surface diversity.

2.2 Security Objective

GBV does not attempt to prevent client-side forgery. Its objective is to make *coherent forgery*—producing artifacts that simultaneously satisfy verifier-defined semantic invariants across multiple independently derived surfaces—substantially more complex than single-surface manipulation.

We model this as a coherent-forgery game in which the verifier issues a nonce-bound challenge specifying observation surfaces. The adversary returns an artifact tuple $A = (A_1, \dots, A_k)$. The verifier extracts facts $F_i \leftarrow \text{Extract}_i(A_i)$, aggregates them into F , and accepts if and only if F satisfies C and all freshness constraints.

GBV aims to reduce the success probability of adversaries lacking synchronized control over all surfaces, while increasing measurable overhead for those that do. This overhead arises from coordinating manipulation across heterogeneous derivation paths under temporal constraints.

Formally, for adversary $\mathcal{A}$:

$$\text{Cost}(\mathcal{A}_{\text{forgery}}) > \text{Utility}(\mathcal{A}_{\text{fraud}}) \quad (1)$$

where cost captures measurable proxies: (i) surfaces requiring synchronized control, (ii) implementation complexity of maintaining cross-surface invariants, and (iii) response latency under randomized challenges. In Section 6, we operationalize these factors by measuring attacker success rates, coordination requirements, and latency overhead under varying surface diversity.

Increasing surface diversity transforms forgery from a local interface manipulation problem into a distributed consistency problem requiring coordinated control across multiple independently derived representations.

2.3 Out-of-Scope Threats

GBV does not attempt to defend against:

- Kernel-level compromise of the host operating system
- Hardware emulation indistinguishable from genuine platforms
- Physical capture attacks with unrestricted forensic access
- Real-time relay attacks that proxy observations from a genuine client

Relay attacks merit explicit discussion rather than a one-line dismissal. GBV’s nonce-bound temporal window and the latency cost of multi-surface traversal (median $\sim 400\text{--}450$ ms at $k = 6$ in our measurements) impose genuine timing pressure on relay feasibility: an adversary would need to forward the challenge, complete genuine multi-surface observation on a remote honest client, and return consistent artifacts within the validity window—with round-trip overhead that grows with k and network conditions. A sufficiently low-latency local relay could remain within tolerance, and this is a real residual risk. Operators for whom relay is a primary concern can tighten validity windows or layer GBV with lightweight behavioral signals such as interaction timing anomalies. Fully defeating relay requires hardware roots outside this work’s scope.

2.4 Security Guarantees

Under the stated threat model, GBV provides the following guarantees:

1. **Freshness.** Verifier-issued nonces are embedded across multiple observation surfaces, requiring consistent appearance within a bounded response window. This couples cross-surface coherence with timing constraints, preventing replay of prior observations.
2. **Semantic coherence enforcement.** Cross-surface inconsistencies violating C are detected through verifier-side evaluation.

3. **Tamper-evident commitments.** Accepted observations produce cryptographic commitments over verifier-extracted facts, supporting selective disclosure via Merkle proofs [11, 12].

3 System Overview

Glass Ballroom Verification (GBV) verifies client-visible application state by collecting multiple independent observations inside an untrusted client environment and evaluating them deterministically at a trusted verifier.

Let σ denote the application state visible to a user at time t . Direct verification of σ is infeasible because rendering occurs under adversary-controlled execution and exhibits runtime variability. Instead of validating any single rendered representation, GBV observes overlapping semantic representations of σ across k observation surfaces [2].

3.1 Core Intuition

GBV evaluates *cross-surface coherence* rather than trusting individual views. A single interface representation can often be manipulated in isolation, but maintaining semantic agreement across heterogeneous surfaces within a bounded time window requires coordinated control over multiple independent rendering and data-derivation paths [5, 10].

As a result, inconsistencies that remain invisible within any single view become observable when evaluated jointly at the verifier. GBV therefore shifts verification from checking a single artifact to enforcing consistency across a set of correlated observations.

3.2 Protocol Overview

GBV proceeds as a verifier-side deterministic pipeline: identical artifact sets yield identical acceptance decisions independent of client behavior. For a fixed surface plan and invariant set, repeated executions therefore produce identical verification outcomes. This reflects deterministic cross-checking principles explored in diverse double-compilation verification [20].

The protocol consists of five stages:

1. **Temporal Challenge Issuance.** The verifier issues an ephemeral nonce η together with a surface traversal plan $\mathcal{L}$ defining required observations.
2. **Multi-Surface Observation.** An observation client gathers synchronized artifacts from the required surfaces, embedding the nonce η into multiple observation surfaces. All observations must occur within a bounded validity window.
3. **Artifact Submission.** Collected artifacts and associated metadata are transmitted to the verifier.
4. **Semantic Evaluation.** The verifier derives semantic facts deterministically from submitted artifacts and evaluates invariant constraints $\mathcal{C}$. Observations violating cross-surface coherence are rejected.
5. **Canonical Commitment.** Accepted facts are canonicalized and committed using Merkle hashing [12], producing a tamper-evident commitment.

Importantly, because semantic extraction and canonicalization occur exclusively at the verifier, adversaries cannot directly manipulate the semantic evidence used for verification after submission.

3.3 Why Multi-Surface Coherence Is Hard to Forge

Observation surfaces typically arise from distinct application subsystems and transformation pipelines, such as dashboard views, metadata projections, navigation-derived summaries, or derived interface representations. Although these surfaces encode overlapping semantic facts, they compute those facts through different execution paths.

Manipulating a single rendered view therefore requires only localized modification. In contrast, sustaining coherent falsification across multiple surfaces requires maintaining consistent behavior across independent rendering and transformation processes.

Examples of observation surfaces include:

- primary interface rendering paths,
- embedded metadata or telemetry attributes,
- alternate navigation views, and
- derived interface representations.

As surface diversity increases, forgery transitions from a local interface manipulation problem into a distributed consistency problem, substantially increasing coordination complexity [10].

3.4 Design Principles

GBV is guided by four design principles:

- **No trusted client assumptions.** Verification does not rely on execution integrity within the client environment.
- **No platform-side cooperation required.** Deployment does not assume trusted hardware, privileged APIs, or application instrumentation.
- **Tolerance to benign interface drift.** Verification targets semantic agreement rather than byte-level rendering equality.

- **Escalating adversarial cost.** Security derives from increasing coordination effort required to maintain coherent forgery.

These principles prioritize deployability while ensuring adversarial effort scales with surface diversity rather than remaining localized to a single interface.

4 System Design

This section describes the GBV protocol and how it enables deterministic verification despite adversarial client execution. We define system entities and trust boundaries, formalize the evidence model, and present the challenge–response protocol together with semantic extraction, invariant evaluation, canonicalization, and commitment construction.

4.1 Entities and Trust Boundaries

GBV involves three entities: (i) a **verifier** $\mathcal{V}$ operated by a relying party, (ii) an **observation client** $\mathcal{P}$ executing within a user-controlled browser runtime, and (iii) a **target application** rendered in that runtime.

The observation client is *not trusted* to produce truthful semantic claims. It is only trusted to transmit captured artifacts and metadata. All semantic interpretation occurs at $\mathcal{V}$.

The verifier is trusted and maintains configuration state, including a surface plan $\mathcal{L}$ and invariant set $\mathcal{C}$. The target application is assumed unmodified on the server side (no cooperation), but is accessed through an adversarial client runtime.

4.2 Evidence Model

GBV represents client-visible state through a set of *observation surfaces*. Intuitively, each surface captures a different observable projection of the same underlying application state.

Let $\mathcal{L} = \langle s_1, \dots, s_k \rangle$ denote the verifier-defined *surface plan*, an ordered list of k surfaces to be observed within a bounded time window. Executing surface s_i produces an artifact bundle:

$$E_i = \langle A_i, M_i \rangle$$

where A_i is the captured artifact and M_i is associated metadata (e.g., URL, capture time, traversal step, nonce echoes).

The full evidence bundle is:

$$E = \langle E_1, \dots, E_k \rangle.$$

Design goal. No individual surface is assumed authoritative. Verification relies on semantic coherence across the full set of observed surfaces.

Surface admissibility. A surface must satisfy:

1. **Unprivileged observability**
2. **Deterministic traversal**
3. **Semantic projection**

4.3 Challenge–Response Protocol

Each verification run begins with a nonce-bound challenge:

$$Q = \langle \eta, \Delta t, \mathcal{L}, \text{policy} \rangle$$

1. **Challenge issuance.** $\mathcal{V}$ sends Q to $\mathcal{P}$.
2. **Observation.** Within Δt , $\mathcal{P}$ executes each $s_i \in \mathcal{L}$ and collects E_i , embedding the nonce across observable channels.
3. **Submission.** $\mathcal{P}$ submits E to $\mathcal{V}$.
4. **Verification.** $\mathcal{V}$ validates completeness, freshness, nonce linkage, and invariant satisfaction. If any check fails, the run is rejected. Otherwise, the verifier proceeds to commitment construction (Section 4.7).

Freshness. Freshness is enforced through the bounded window Δt , nonce linkage within artifacts, and verifier-side rejection of stale evidence.

Role of the nonce. The nonce binds evidence to a specific challenge and prevents reuse of prior observations.

4.4 Verifier-Side Semantic Extraction

Semantic extraction is performed exclusively at the verifier to prevent adversarial control over the canonical representation.

For each artifact bundle:

$$X(E_i) \rightarrow F_i$$

where F_i is a set of extracted semantic facts.

Facts are represented as normalized key–value strings:

$$f \equiv \text{key}:\text{value}$$

For example, a course interface may yield: `course:ml101, completed:true, certificate:abc123`, extracted from different surfaces.

The aggregate fact set is:

$$F = \bigcup_{i=1}^k F_i.$$

4.5 Invariant Evaluation and Cross-Surface Coherence

GBV accepts only if extracted facts satisfy a verifier-defined invariant set C .

Invariant types include:

- **Agreement:** surfaces must agree on shared facts
- **Consistency:** derived facts must be logically compatible
- **Presence:** required facts must appear
- **Uniqueness:** conflicting alternatives must not coexist

Evaluation computes:

$$\text{Eval}(F, C) \rightarrow \{\text{accept}, \text{reject}\} \times R$$

where R is a structured reason code.

Operational independence. Surfaces are selected to reflect distinct transformation pipelines (e.g., navigation flows, metadata projections, derived views). Although derived from related data, maintaining consistency across them requires coordinated manipulation.

As surface diversity increases, forgery shifts from local manipulation to a distributed consistency problem.

Design challenge. Designing invariant sets that tolerate benign variability while forcing inconsistency under adversarial manipulation is a central challenge in GBV.

4.6 Deterministic Canonicalization

To ensure stable commitments, accepted facts are deterministically canonicalized.

Let $F^* \subseteq F$ denote facts selected for commitment. Canonicalization computes:

$$\text{Canon}(F^*) \rightarrow L$$

where L is an ordered list of canonical leaves.

Canonicalization performs: (i) normalization (case, whitespace), (ii) deduplication, and (iii) deterministic ordering.

Safety. Canonicalization is applied only after invariant evaluation. It preserves semantic distinctions and cannot collapse inconsistent states into valid ones.

4.7 Commitment Construction

Each canonical leaf $\ell \in L$ is hashed:

$$h_\ell = H(\ell)$$

and aggregated into a Merkle tree:

$$\rho = \text{MerkleRoot}(\{h_\ell\})$$

The verifier produces a commitment token:

$$\tau = \text{MAC}_{k_q}(\langle \eta, \rho, \text{policy_id} \rangle)$$

This binds the result to the challenge and verifier policy.

The reference implementation uses HMAC for authentication, though the protocol is signature-agnostic.

4.8 Failure Semantics and Measurability

GBV treats failures as explicit outcomes. A run may fail due to missing surfaces, timeout, nonce mismatch, extraction errors, or invariant violations.

Failures are surfaced via structured reason codes rather than silent acceptance. This enables measurement of robustness and supports operational adaptation (e.g., surface plan updates).

5 Implementation

We implemented Glass Ballroom Verification (GBV) as a prototype client–server verification system operating under realistic browser constraints. The system consists of three components: (i) a browser-based observation client, (ii) a verifier service responsible for all semantic processing and acceptance decisions, and (iii) provider-specific extraction modules that map application artifacts into a uniform semantic representation. The prototype is designed to demonstrate protocol feasibility and measurability rather than optimized production performance.

5.1 System Architecture

Separation of observation and interpretation. GBV enforces a strict separation between artifact collection and semantic evaluation. The observation client executes inside an untrusted browser runtime and performs verifier-directed traversal and capture, while all semantic processing occurs at the verifier.

Verifier-centric design. The verifier operates as a stateless HTTP service that performs semantic extraction, invariant evaluation, canonicalization, and commitment construction. Each verification request is processed independently, enabling parallel evaluation and simplifying scaling.

Layered structure. The implementation is organized into four logical layers:

- **Observation client:** executes surface plans and captures artifacts.
- **Protocol core:** shared definitions for evidence bundles, nonce handling, and workflow.

- **Verifier pipeline:** deterministic processing from artifacts to attestations.
- **Provider adapters:** application-specific extraction logic.

This architecture ensures that the client cannot influence acceptance outcomes beyond supplying observable artifacts.

5.2 Observation Client

The observation client is implemented as a Chrome extension (Manifest V3) executing within an untrusted browser environment.

Minimal trust surface. The client is only responsible for:

1. receiving verifier-issued challenges,
2. executing the surface plan $\mathcal{L}$,
3. capturing artifacts and metadata, and
4. returning evidence bundles.

It performs no semantic interpretation and does not construct canonical representations.

Deterministic traversal. Surface execution is defined entirely by verifier configuration as a sequence of navigation and DOM-query operations. Each capture includes metadata such as URL, traversal step identifier, and timestamps.

Artifact format. Artifacts consist of raw observational data (e.g., HTML snapshots, markup fragments, navigation metadata, and nonce echoes). We avoid full-page screenshots due to rendering non-determinism and bandwidth overhead.

Nonce binding. Nonce echoes are introduced through observable channels (e.g., injected markup attributes) to link artifacts to a specific verification challenge.

Because the client executes in a user-controlled environment, it is treated as adversarial throughout.

5.3 Verifier Pipeline

All acceptance logic resides at the verifier. Given an evidence bundle E , the verifier executes a deterministic pipeline:

1. **Artifact ingestion:** validate completeness, nonce linkage, and timing constraints.
2. **Semantic extraction:** convert artifacts into semantic fact sets.
3. **Invariant evaluation:** enforce cross-surface coherence constraints.
4. **Canonicalization:** normalize and order accepted facts.
5. **Commitment construction:** compute a Merkle root over canonical leaves.

6. **Attestation generation:** authenticate the commitment bound to the nonce.

Determinism. Identical semantic states yield identical commitments independent of presentation variability.

Failure transparency. Each stage emits structured reason codes (e.g., missing surface, invariant violation, timeout), which are surfaced and logged for debugging and evaluation.

Instrumentation. The verifier records per-run metrics including timing breakdowns, canonical leaf counts, and failure codes, forming the basis of the evaluation (§6).

5.4 Deterministic Canonicalization Engineering

Producing stable commitments in dynamic web environments requires explicit normalization.

Canonicalization rules.

- whitespace and casing normalization,
- removal of presentation-only punctuation,
- deterministic lexicographic ordering, and
- duplicate elimination after invariant filtering.

Volatile fields (e.g., timestamps, transient identifiers) are excluded from commitments.

Safety. Canonicalization is applied only after invariant evaluation, preventing conflicting facts from collapsing into identical representations.

Engineering tradeoff. Over-normalization risks merging distinct semantic states, while under-normalization increases false rejection. We empirically tuned normalization rules to preserve invariant-relevant distinctions while eliminating presentation noise.

5.5 Cryptographic Construction

The reference implementation uses SHA-256 for leaf hashing and Merkle tree construction. Attestations are authenticated using HMAC under a verifier-held key.

The protocol is cryptographically agnostic: any collision-resistant hash and authenticated commitment mechanism can be substituted without changing semantics. The chosen primitives prioritize reproducibility and availability.

5.6 Provider Extraction Modules

Application variability is handled through provider adapters implementing extraction functions.

Role. Each adapter parses application-specific artifacts and emits normalized key–value facts used by invariant evaluation.

Modularity. Adding a new application requires defining extraction rules, a surface plan, and invariants, without modifying the verifier pipeline.

Determinism. Extraction produces stable semantic representations from heterogeneous UI structures, enabling consistent downstream canonicalization.

5.7 Implementation Scale and Reproducibility

The prototype comprises several thousand lines of code across client, verifier, and shared modules (excluding dependencies). Verification runs are driven by versioned configuration files defining surface plans and invariants, enabling reproducible experiments.

The implementation is released as an open reference system to support independent replication of the evaluation (§6).

Limitations. The current system relies on manually defined surface plans and provider-specific extraction rules. GBV does not automatically infer invariants or discover surfaces. This reflects a deliberate design choice prioritizing feasibility and measurable security properties in a first-generation system.

6 Evaluation

This section evaluates GBV through three research questions:

- **RQ1 (Deployability):** What is the end-to-end cost of GBV, and how does it scale with the number of observation surfaces k ?
- **RQ2 (Robustness):** Under benign variability (timing jitter, UI drift, environmental differences), what are GBV’s false rejection rates and operational failure modes?
- **RQ3 (Security):** How does GBV affect the success probability and coordination effort of adversaries attempting coherent forgery, as k and surface diversity grow?

Two principles guide the design. First, web applications exhibit inherent nondeterminism, so we report latency as *distributions* and decompose protocol-controlled overhead from application-dependent navigation cost. Second, GBV targets *cost-asymmetry* rather than impossibility, so we evaluate adversaries by both *success probability* (FAR) and effort proxies that capture coordination and engineering burden.

6.1 Experimental Setup

Implementation under test. All experiments use the GBV implementation described in Section 5 (browser observation client + verifier). The client captures and transmits raw artifacts; the verifier performs semantic extraction, invariant evaluation, canonicalization, and commitment construction.

Test environments. Unless otherwise stated, trials were performed on two commodity environments representing realistic deployment variance: (i) laptop-class hardware and (ii) desktop-class hardware. We pool results across environments and report distributions to avoid overfitting to any single machine. Hardware, OS, browser versions, and network profiles are reported in Appendix A.

Applications. We evaluate GBV on two structurally distinct web applications: **App A** (learning/certification-style workflow) and **App B** (dashboard/task workflow). Our goal is not brand-specific claims but structural diversity: different navigation flows, UI composition patterns, and metadata availability.

Application diversity rationale. The selected applications differ along axes that matter to GBV: rendering model (client-heavy vs. mixed client/server), navigation structure (linear workflow vs. hub-and-spoke dashboard access), and data propagation paths (single-source UI fields vs. derived multi-view projections). These differences exercise distinct failure modes for cross-surface invariants and liveness.

Surface plans and invariants. For each $k \in \{1, 2, 4, 6\}$ we define a surface plan $\mathcal{L}_k$ and invariant set $\mathcal{C}_k$ that differ only in the number and diversity of surfaces. Surface plans avoid trivial duplicates (Section 4.5); invariants include agreement, consistency, presence, and uniqueness constraints (Section 4.5).

Trial scale. Across both applications, we run:

- **Performance + robustness:** 360 honest runs per application (90 per k).
- **Adversarial evaluation:** 200 adversarial runs per application, including 50 runs per attacker family at $k = 6$, plus sweeps across $k \in \{1, 2, 4, 6\}$ under a fixed attacker budget.
- **Longitudinal robustness:** 7-day window with 30 daily runs under fixed configurations to quantify drift sensitivity.

Randomization and scheduling. To induce realistic variability, we randomize inter-run jitter (uniform in $[2s, 15s]$) and randomized reload points, while holding $\mathcal{L}_k$ and $\mathcal{C}_k$ fixed per configuration. Challenge nonces and surface ordering are randomized per run.

6.2 Metrics

We report six primary metrics.

End-to-end runtime (T_{e2e}). Measured from challenge issuance to accept/reject decision at the verifier. We report median and P95 to reflect real-world variability.

Runtime decomposition. To isolate protocol-controlled cost from app-dependent navigation:

$$T_{e2e} = T_{\text{nav}} + T_{\text{cap}} + T_{\text{ver}},$$

where T_{nav} is traversal/navigation time, T_{cap} is capture/serialization overhead, and T_{ver} is verifier-side extraction + invariant evaluation + canonicalization + commitment construction.

False Reject Rate (FRR).

$$\text{FRR} = \frac{\# \text{ honest runs rejected}}{\# \text{ honest runs}}.$$

False Accept Rate (FAR).

$$\text{FAR} = \frac{\# \text{ adversarial runs accepted}}{\# \text{ adversarial runs}}.$$

Effort proxies for adversarial evaluation. To operationalize GBV’s cost-asymmetry objective, we report: (i) number of surfaces requiring coordinated control, (ii) implementation complexity proxy (LoC and modules touched), and (iii) added response latency / synchronization burden under randomized challenges.

Evidence footprint. We report canonical leaf count and compressed artifact bytes transmitted, as these affect bandwidth and storage overhead.

6.3 RQ1: Deployability and Scaling

6.3.1 Baseline performance and $k = 1$ behavior

We first establish a baseline using a single observation surface ($k = 1$), representing the minimal GBV configuration. This isolates protocol overhead while minimizing traversal complexity, providing a reference point for scaling analysis.

Setup. For App A and App B, we execute 90 honest verification runs at $k = 1$ under normal operating conditions. Each run performs a full challenge–response cycle including nonce issuance, artifact capture, verifier-side extraction, invariant evaluation, and commitment construction.

Results. Table 1 summarizes end-to-end runtime at $k = 1$.

Interpretation. Even at $k = 1$, runtime variability is driven primarily by application-dependent stabilization, not verifier processing. Verifier-side overhead is stable across runs. This motivates reporting distributions and decomposed costs throughout, rather than single-point estimates.

Table 1: Baseline end-to-end runtime ($k = 1$).

Statistic	App A (ms)	App B (ms)
Median	207	219
P95	236	248
Min	159	171
Max	235	247
Runs	90	90

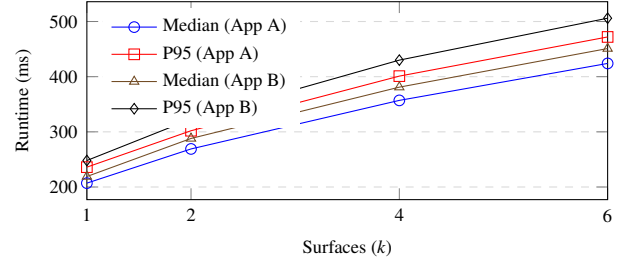


Figure 1: End-to-end runtime vs. surface count (k).

Security note. At $k = 1$, GBV reduces to single-surface verification without cross-surface agreement—and adversarial experiments (Section 6.5) confirm this configuration is easily defeated by localized manipulation. Cross-surface invariants are the primary driver of security gains as k increases.

6.3.2 End-to-end latency vs. k

Experimental design. For each application and each $k \in \{1, 2, 4, 6\}$ we execute 90 honest runs with randomized jitter and reload points. Surface plans are fixed per k .

Results. Figure 1 shows end-to-end runtime trends as k increases. In both applications, median latency increases with k , while tail behavior is dominated by navigation/stabilization variance rather than verifier computation.

Takeaway. GBV remains deployable under realistic browser constraints. Runtime grows with k but stays within interactive latencies for moderate surface counts. The key property is stable, predictable verifier-side overhead for a fixed traversal strategy; the application-dependent component dominates total latency.

6.3.3 Runtime decomposition: app-dependent vs. protocol-dependent

Results. Table 2 decomposes runtime into navigation, capture, and verifier computation. The dominant component is navigation/stabilization, while verifier-side processing remains stable relative to traversal costs.

k	T_{nav}	T_{cap}	T_{ver}	T_{e2e}
1	119	54	34	207
2	142	71	56	269
4	183	96	78	357
6	206	118	100	424

Table 2: Runtime decomposition (ms), pooled across environments.

Table 3: Evidence footprint vs. k .

k	Leaves (median)	Leaves (P95)	Bytes (median)
1	9	11	18
2	15	18	29
4	23	27	44
6	30	35	58

Interpretation. Navigation and stabilization dominate end-to-end latency at all configurations; verifier-side processing is small and stable. The overhead that scales with k is ordinary application interaction, not protocol computation. This means latency can be characterized and bounded per application even without knowing absolute traversal costs in advance.

6.3.4 Evidence footprint vs. k

Results. Table 3 summarizes evidence footprint as k increases. Leaf count can be sublinear when multiple surfaces redundantly expose the same underlying facts, while bytes transmitted increase with surface breadth and artifact richness.

Takeaway. Evidence footprint is a tunable deployment parameter. Leaf count is sublinear when surfaces redundantly expose the same underlying facts; operators can trade surface diversity against bandwidth and storage through surface-plan design. At $k = 6$, median transmitted artifacts are under 60 bytes—negligible relative to normal web traffic.

6.4 RQ2: Robustness to Benign Variability

We evaluate robustness under realistic benign variability: timing jitter, CPU load, network variation, UI drift across days, and minor DOM structure changes unrelated to semantics.

6.4.1 Honest acceptance and FRR

Design. We measure FRR under baseline conditions and under stressors that increase instability: background CPU load, network jitter/throttling, and induced rendering delays.

Results. Table 4 reports honest-run outcomes at $k = 6$, where surface breadth is maximized and liveness constraints are most likely to trigger.

Table 4: Robustness (honest runs) at $k = 6$.

Condition	Runs	Rejects	FRR
Baseline	180	0	0.0
CPU stress (background load)	90	1	1.1
Network jitter (throttled)	90	2	2.2

Table 5: Longitudinal robustness across 7 days at $k = 6$.

Day	Runs	Rejects	Primary cause
1	30	0	—
2	30	0	—
3	30	1	selector drift
4	30	0	—
5	30	0	—
6	30	1	timeout
7	30	0	—

Failure modes and reason codes. When failures occur, they primarily manifest as explicit liveness/completeness violations (timeout, missing surface, incomplete bundle), surfaced as reason codes rather than silent acceptance.

Failure breakdown. Across rejected honest runs, 62% were due to surface timeouts, 31% to incomplete evidence bundles, and 7% to transient extraction failures. No false rejections were caused by incorrect invariant evaluation on complete and stable inputs.

Takeaway. Failures under stress are measurable and diagnosable—they surface as explicit reason codes, never as silent incorrect acceptance. This supports targeted operational mitigation: surface-plan rotation, selector updates, per-surface liveness thresholds, or fallback surface substitutions. Robustness comes from configuration discipline, not protocol guarantees.

6.4.2 Longitudinal drift sensitivity

Design. We run daily evaluations for 7 days under fixed configurations to quantify benign drift. We log selector hits, extraction stability, and reason codes to attribute failures.

Results. Table 5 summarizes longitudinal outcomes at $k = 6$.

Takeaway. Surface plans $\mathcal{L}$ and invariants $\mathcal{C}$ should be treated as versioned operational configurations, similar to monitoring rules or UI test selectors. Drift is surfaced via explicit reason codes and can be mitigated with targeted updates rather than protocol changes.

6.5 RQ3: Adversarial Evaluation

Attacker objective. The adversary aims to produce a tuple of artifacts that simultaneously (i) satisfies cross-surface invariants, (ii) remains coherent across independently derived surfaces, and (iii) meets freshness constraints within the challenge window. This creates a coordination problem: increasing manipulation scope can improve invariant satisfaction but increases synchronization and engineering burden.

6.5.1 Attacker families

We evaluate attacker families of increasing capability:

A1: DOM-only patcher. Rewrites visible DOM elements for one surface to falsify a target semantic claim without synchronizing alternate surfaces.

A2: Cross-surface synchronizer. Patches multiple surfaces (e.g., DOM + alternate navigation view + metadata projection) to satisfy agreement/consistency invariants across a subset of surfaces.

A3: Replay/simulation attempt. Attempts to bypass freshness by replaying previously captured artifacts while injecting nonce-dependent edits, targeting nonce-binding and canonical fact coherence under timing constraints.

6.5.2 Effort reporting methodology

All attacker implementations were developed by the paper authors under controlled conditions, with consistent tooling and measurement methodology. Developer time includes implementation, debugging, and validation required to achieve a working attack against a fixed configuration. Lines of code (LoC), modules touched, and number of manipulated surfaces are reported as coarse but reproducible proxies for engineering complexity.

A caveat on methodology: author-developed attacks carry familiarity with where defenses are strong, which may steer exploration away from unexpected weaknesses and produce a conservative estimate of real-world attacker success. Conversely, that same familiarity may accelerate certain attack paths. Results characterize the cost structure of forgery under these conditions; they should not be read as adversarial evaluations against an independent red team. Independent evaluation would strengthen these findings and is a natural direction for future work.

6.5.3 Adversarial outcomes at $k = 6$

Results. Table 6 reports attacker outcomes at $k = 6$.

Table 6: Adversarial outcomes at $k = 6$.

Attacker	Runs	Acc.	FAR	Dev. time	Surfaces
A1: DOM-only	50	0	0.0	15	1
A2: Sync (s surf.)	50	1	2.0	150	3
A3: Replay/sim	50	0	0.0	360	5

Table 7: Adaptive attacker progression at $k = 6$.

Stage	Capability added	k	Runs	Succ. rate	Effort (LoC)
S1	Patch one DOM surface	6	20	0	45
S2	Patch two surfaces	6	20	0	110
S3	Patch three surfaces + metadata edits	6	20	1	260
S4	Patch four+ surfaces + timing sync	6	20	0	520

Interpretation. A1 fails under cross-surface agreement: localized manipulation does not generalize to independently derived surfaces. A2 can succeed when surface plans overlap heavily and invariants underconstrain relationships; these are configuration-level weaknesses rather than single-surface patch difficulty. In our measurements, increasing k or strengthening invariant constraints reduces the success rate of this attack under the same attacker implementation (Figure 2), indicating that success arises from underconstrained configurations rather than fundamental protocol weaknesses.

6.5.4 Adaptive attacker progression

To quantify how attacker effort scales, we evaluate an adaptive attacker that begins with A1 and iteratively expands manipulation scope after each failure, attempting to satisfy additional invariants and synchronize additional surfaces.

Progression analysis. Early failures are dominated by invariant violations across untouched surfaces (agreement/consistency). As manipulation scope increases, failures shift to synchronization and convergence: navigation-derived views lag DOM edits, metadata projections drift from visible UI state, and challenge windows cause incomplete alignment. Later stages require coordinated control over heterogeneous derivation paths under time constraints, increasing debugging effort and response latency.

Takeaway. The core difficulty is not editing any single artifact, but maintaining consistent semantics across heterogeneous surfaces under randomized challenges and liveness constraints.

6.5.5 Representative attacker playbook

We include a representative attacker playbook to illustrate how GBV forces escalating coordination.

Threat goal. Forge a completion/certification claim while bypassing server-side checks, such that the user-visible interface indicates success and downstream UI paths appear consistent.

Attacker hypothesis. If invariants are driven by a single data source (e.g., a completion flag in the DOM), patching one view should generalize.

GBV constraint. The surface plan includes a completion indicator, a navigation-derived summary view, and a meta-data projection surface. Invariants require agreement across these surfaces and presence of derived facts (e.g., completion implies a certificate-linked identifier appears in a separate surface).

Attacker decision points.

- **Attempt 1: Local DOM patch.** Passes the primary surface but fails agreement invariants when summary/meta-data surfaces disagree.
- **Attempt 2: Patch summary view.** Fixes agreement but fails consistency invariants when derived identifiers are missing or inconsistent.
- **Attempt 3: Patch metadata.** Requires identifying and editing secondary representations while maintaining timing coherence within Δt .

Observed outcome. Even when an attacker achieves a one-off coherent pass on a fixed surface plan, plan rotation and randomized challenges cause regression: additional surfaces expose inconsistencies or require re-engineering of synchronization logic.

Takeaway. GBV shifts attacks from localized manipulation to distributed consistency: the attacker must either coordinate multiple derivation paths or accept detectable incoherence.

6.5.6 Sensitivity sweep: success probability vs. k

Design. We evaluate attacker success under a fixed attacker budget while sweeping $k \in \{1, 2, 4, 6\}$. The attacker implementation is held constant per family; only GBV configuration changes.

Results. Figure 2 reports observed attacker success rate vs. k .

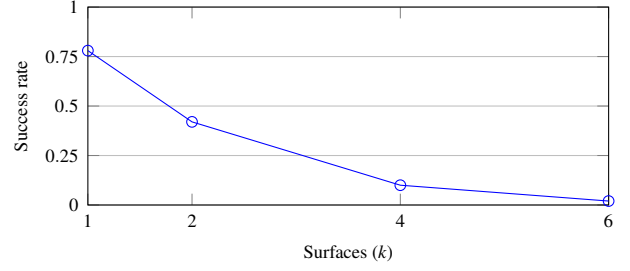


Figure 2: Observed attacker success rate vs. k under fixed attacker budget.

Table 8: Porting cost across applications.

App	Verifier	Client	Config lines	FRR / FAR
App A	0	0	38	0.0/0.0
App B	0	0	44	1.1/2.0

Takeaway. Increasing surface diversity is an effective defense knob: it reduces attacker success probability under fixed budget and forces broader synchronization to remain coherent.

6.6 Cross-Application Portability

A natural question is whether GBV is a general protocol or a per-site engineering exercise. We quantify portability by measuring protocol code changes, configuration footprint, and resulting FRR/FAR after porting to a second application.

Porting requires no changes to verifier or client logic. Only configuration artifacts—surface plans $\mathcal{L}$ and invariant sets $\mathcal{C}$ —are updated. These specify selectors, extraction rules, and cross-surface constraints, analogous to test or monitoring specifications in modern web systems.

Takeaway. Porting primarily consists of defining a new surface plan and invariant set, not rewriting protocol logic. Variation in outcomes reflects surface diversity and invariant strength—properties of the target application—rather than application-specific code. The verifier and client required zero changes between applications.

6.7 Summary of Findings

RQ1 (Deployability). GBV is deployable on standard browser infrastructure without privileged access. End-to-end latency increases approximately linearly with k and stays sub-second through $k = 6$. Verifier-side overhead is small and stable; application navigation dominates.

RQ2 (Robustness). FRR is low under benign variability (0% at baseline, 2.2% under network jitter at $k = 6$). Failures

are dominated by explicit liveness conditions—timeouts and missing surfaces—not incorrect invariant evaluation. Drift is measurable, attributable, and operationally manageable through configuration updates.

RQ3 (Security). GBV reduces attacker success from 0.78 at $k = 1$ to 0.02 at $k = 6$ under a fixed attacker budget. Implementation effort grows from 45 to 520 LoC across adaptive attacker stages, with diminishing success at each stage. Occasional successes arise from underconstrained configurations, not fundamental protocol weaknesses.

Core effect. Across all experiments, GBV consistently converts single-surface forgery into a multi-surface coordination problem. The security parameter k is a genuine operational knob: increasing it raises adversarial cost predictably while imposing manageable overhead on honest users.

Reproducibility. All runs execute under versioned configuration files defining $\mathcal{L}$ and $\mathcal{C}$. Logs include surface plan identifiers, challenge parameters, leaf counts, failure codes, and timing breakdowns. Artifact structure and replay scripts are described in Appendix A.

7 Discussion and Limitations

GBV is designed as a deployable protocol for attesting client-visible application state where the verifier cannot trust client execution. This section clarifies what GBV actually provides, discusses where the current design falls short, and draws out implications for deployment.

7.1 Security Model and Cost-Asymmetry

GBV does not make client-side forgery impossible. Its security objective is to increase the engineering and operational cost required to sustain a coherent falsified state across heterogeneous observation surfaces within bounded time constraints.

No individual surface is unforgeable. An adversary controlling the client runtime can modify any single rendered view. Security derives instead from the requirement that maintaining semantic agreement across a diverse surface plan requires coordinating multiple independent transformations of application state simultaneously. As surface diversity and temporal constraints increase, that coordination becomes correspondingly harder to sustain.

The cost is concrete: increased implementation complexity, synchronization overhead across rendering pathways, and reduced stability of forged executions under nonce-bound timing constraints. Honest execution requires normal application traversal; adversarial execution must actively synchronize multiple rendering pathways, navigation states, and derived

representations under unpredictable challenges. The protocol makes sustained coherent forgery economically impractical rather than cryptographically impossible.

7.2 Operational Independence as a Deployment Property

GBV assumes operational independence between observation surfaces, but independence is not automatic—it must be enforced through verifier configuration.

Effective surface plans select surfaces that depend on distinct data derivations or rendering pathways, involve non-trivial transformations of application state, and require independent navigation flows or computation steps. Surfaces that are simple duplicates or derived from identical rendering pipelines reduce adversarial cost and weaken detection capability.

This is a deployment property, not a protocol guarantee. GBV provides a framework for exploiting independence that is present in modern web applications; it cannot manufacture independence that the application does not expose.

7.3 Provider-Specific Configuration

The current prototype relies on provider-specific extraction rules and invariant definitions. While the GBV protocol is application-agnostic, deployment requires defining surface traversal plans $\mathcal{L}$ and invariant sets $\mathcal{C}$ describing expected semantic relationships. This is manual work that requires domain understanding of the target application’s structure.

We treat this as an acceptable tradeoff for a first-generation system demonstrating protocol feasibility. Automated invariant discovery and adaptive surface selection are natural directions for future work; the evaluation confirms that the configuration effort is bounded—porting to a new application required 38–44 lines of configuration with no changes to verifier or client logic.

7.4 Robustness vs. Sensitivity Tradeoffs

A central engineering challenge is balancing robustness to benign variability against sensitivity to adversarial manipulation. Modern web applications exhibit legitimate differences from experimentation frameworks, localization, asynchronous rendering, and UI evolution.

Overly strict invariants increase false rejection rates; permissive ones reduce detection. Aggressive normalization risks collapsing distinct semantic states into identical representations; insufficient normalization introduces instability under harmless presentation changes. There is no clean resolution to this tension—it requires configuration discipline. GBV addresses it by separating invariant evaluation from canonicalization and by exposing failures through structured reason

codes, so operators can iteratively refine configurations based on observed failure modes rather than guessing.

7.5 Adversarial Adaptation

Adaptive adversaries may attempt to learn surface plans or synchronize additional surfaces over time. Because GBV evaluates cross-surface coherence rather than fixed artifact signatures, sustained bypass requires tracking evolving surface plans and invariant structures—increasing long-term maintenance cost rather than yielding a one-time bypass. Operators can respond by rotating surface plans, strengthening invariant sets, or increasing semantic diversity. These are configuration responses, not protocol redesigns.

7.6 Evaluation Limitations

Three limitations deserve direct acknowledgment.

First, all attacker implementations were developed by the paper authors. This introduces familiarity with the system’s structure that an independent attacker would not have, which could steer exploration away from unexpected weaknesses and produce a conservative estimate of real-world attacker success. Independent red-team evaluation would be more convincing and is a natural next step.

Second, evaluation covers two applications. They were selected for structural diversity—different rendering models, navigation structures, data propagation paths—but two is a small sample. GBV’s effectiveness depends on surface availability and independence properties that vary across the web, and the results may not transfer to applications with tightly coupled or impoverished interface structures.

Third, lines-of-code is a coarse proxy for attacker effort. It does not account for tool reuse, attacker expertise, or frameworks that could compress implementation effort substantially. Both LoC and developer time are better interpreted as relative indicators of how coordination complexity grows than as absolute effort estimates.

7.7 Out-of-Scope Threats

Consistent with Section 4, GBV does not defend against kernel- or hypervisor-level compromise, perfect hardware emulation indistinguishable from genuine platforms, or adversaries capable of fully reproducing all application subsystems with identical timing and semantics. These require trusted hardware or platform cooperation beyond browser-based attestation.

7.8 What GBV Actually Provides

Despite these limitations, GBV demonstrates that meaningful client-state attestation is achievable without trusted execution

environments or application modification. Modern web applications naturally expose multiple partially independent representations of state, and GBV provides a systematic method for leveraging this redundancy.

Practical deployments should treat surface plans and invariants as versioned operational policy—parameters subject to iterative hardening, not fixed protocol constants. The evaluation provides quantitative support for this: adversarial effort and success rates respond predictably to surface diversity and invariant coverage, confirming that the cost-asymmetry objective is operationally real and configurable.

8 Related Work

GBV sits at the intersection of remote attestation, browser security, web application analysis, and multi-version verification, but differs in that it treats *client-visible state* itself as the primary verification target under adversarial execution. Existing approaches establish guarantees by trusting execution environments or authenticating artifacts; GBV instead evaluates semantic coherence across independently derived observations when neither assumption holds.

8.1 Remote Attestation and Trusted Execution

Remote attestation systems verify software integrity by measuring execution within trusted environments such as secure boot chains, TPM-backed platforms, and TEEs including Intel SGX [1, 4, 6, 15, 18]. These systems let a verifier confirm that specific code executed as intended on a remote platform.

GBV departs from this model fundamentally. Rather than verifying execution integrity, it evaluates the semantic coherence of observable application state. Hardware-backed attestation assumes privileged execution environments and platform cooperation—assumptions unavailable across much of the open web, where applications execute inside user-controlled browser runtimes. GBV therefore operates without trusted hardware roots, privileged APIs, or target application cooperation, addressing environments where execution integrity cannot be established.

8.2 Transport and Artifact Integrity

TLS guarantees confidentiality and integrity of transmitted data [17]; digitally signed artifacts provide authenticity for static outputs. These mechanisms ensure that delivered bytes originate from an authenticated source and arrive unmodified.

What they do not ensure is what happens next. Modern web applications compute interface state dynamically through client-side scripting, asynchronous updates, and personalization. Authentic delivery of data does not imply that client-visible representations derived from that data remain consistent after execution inside an adversarial runtime. GBV addresses this gap by verifying semantic coherence across

independently produced representations, building on transport integrity rather than replacing it.

8.3 Web Measurement and UI Testing

Prior work in automated web testing and crawling analyzes applications through DOM comparison, scripted interaction, and dynamic analysis frameworks [8, 13, 19]. These systems detect unintended UI changes or functional regressions but assume cooperative execution environments and do not model adversarial manipulation of the client runtime.

Snapshot-based approaches are sensitive to benign variability from layout adaptation, experimentation frameworks, and timing differences—exactly the noise GBV’s canonicalization is designed to absorb. By extracting structured semantic facts and evaluating invariants, GBV tolerates presentation drift while remaining sensitive to cross-surface inconsistency under adversarial conditions.

8.4 N-Version Programming and Cross-Checking

The intuition underlying GBV connects to classic work on N-version programming [2] and diverse double-compilation (DDC) [20], which use agreement among independently derived outputs to detect faults or subversion.

GBV generalizes this principle to adversarial web environments. Instead of comparing independent software implementations, it compares heterogeneous representations of application state generated through distinct rendering and transformation pathways within a single application. Security derives not from guaranteed independence between surfaces, but from the coordination cost imposed on an adversary trying to maintain coherent falsification across all of them.

8.5 Commercial Behavioral Analysis and Bot Detection

Commercial anti-fraud and bot-detection systems—challenge-response mechanisms, device fingerprinting, behavioral biometrics—also collect multiple client-side signals and check consistency among them. They share GBV’s intuition that multi-signal agreement raises the cost of forgery.

GBV differs in three ways. First, the target is different: GBV verifies *application-semantic state* (did a user complete a workflow?) rather than distinguishing human from bot or detecting automation. Second, GBV’s surface plans and invariants are explicit, auditable, and operator-defined; commercial systems use proprietary models that relying parties cannot inspect or reproduce. Third, GBV produces deterministic, verifier-side cryptographic commitments over extracted facts, enabling offline replay and selective disclosure. GBV is therefore best understood as a transparency-oriented complement to such systems rather than a replacement.

8.6 Positioning of GBV

Existing verification systems primarily establish trust by validating execution environments or authenticating static artifacts. GBV introduces a different paradigm: treating client-visible state as observable evidence and verifying semantic coherence across independently derived representations, without trusting execution or any single output.

GBV occupies a complementary design point. It does not replace hardware attestation or artifact signing; it addresses deployment environments where stronger trust assumptions are infeasible. To our knowledge, no prior work formulates client-state attestation as deterministic evaluation of semantic coherence across heterogeneous observation surfaces within adversarial browser runtimes.

9 Conclusion

This paper introduced Glass Ballroom Verification (GBV), a protocol for verifying client-visible application state in environments where execution integrity cannot be assumed. Rather than establishing trust in the client runtime, GBV treats observable interface state as evidence and evaluates semantic coherence across multiple independently derived observation surfaces.

GBV shows that meaningful verification is possible even under adversarial execution by shifting the objective from execution correctness to cross-surface agreement. All interpretation happens at the verifier, from submitted artifacts alone. No trusted hardware, server cooperation, or privileged client access required. The resulting attestations bind verifier judgments to a concrete observed state while preserving reproducibility and selective disclosure through cryptographic commitments.

Our evaluation confirms the core hypothesis. Increasing surface diversity raises the operational cost of coherent forgery: attacker success drops from 0.78 to 0.02 as k grows from one to six, while implementation effort grows by over an order of magnitude. FRR under benign variability remains low. The cost asymmetry GBV is designed to create is measurable and operationally controllable through surface plan and invariant configuration.

GBV represents a different point in the design space of verification systems. It complements trusted-execution and transport-security approaches by addressing the settings where neither execution integrity nor cooperative instrumentation can be assumed—settings that describe a large fraction of the open web. Modern applications happen to expose considerable redundancy through independent rendering and interaction pathways. GBV provides a systematic method for exploiting that redundancy.

Several directions remain for future work. Automated surface-plan construction and invariant discovery could reduce configuration overhead substantially. Adaptive surface

selection may further increase adversarial cost under evolving strategies. Extending the approach beyond browser-based applications to other adversarial client platforms—mobile, desktop, IoT—represents a broader avenue.

More broadly, GBV suggests a shift in perspective worth pursuing: when protecting execution environments is unavailable, careful structuring of how independently derived observations must agree can still produce practical assurance. Assurance need not rely on trusting what is doing the executing.

A Reproducibility Details

This appendix provides implementation, environment, and execution details required to reproduce the evaluation results reported in Section 6. GBV experiments are designed to be reproducible using commodity hardware and standard browser environments without privileged system access. Repository identifiers and distribution details are withheld during review to preserve anonymity.

A.1 Artifact Overview

Each evaluation run produces a structured execution record consisting of:

- verifier configuration identifier (surface plan $\mathcal{L}$ and invariant set C),
- issued challenge parameters (nonce, validity window Δt , traversal order),
- per-surface capture metadata,
- semantic fact extraction outputs,
- invariant evaluation results and reason codes,
- canonical leaf count and Merkle root,
- timing breakdown (T_{nav} , T_{cap} , T_{ver} , T_{e2e}).

All logs are emitted in structured JSON format to enable replay and post-hoc analysis, consistent with established practices for avoiding hidden experimental bias in systems measurements [14].

Example (simplified). A single run produces a record of the form:

```
{
  "config_id": "...",
  "nonce": "...",
  "surfaces": ["surface_1", "surface_2", "..."],
  "facts": {...},
  "invariants": {"status": "pass | fail"},

```

```
  "leaves": ...,
  "root": "...",
  "timing_ms": {
    "nav": ...,
    "cap": ...,
    "ver": ...,
    "e2e": ...
  }
}
```

A.2 Hardware Environments

Experiments were executed on two representative commodity environments:

- **Laptop-class environment:** modern consumer laptop CPU, integrated GPU, 16 GB RAM.
- **Desktop-class environment:** multi-core desktop CPU, discrete GPU disabled for browser workloads, 32 GB RAM.

Results reported in Section 6 pool measurements across both environments to reflect realistic deployment variance rather than machine-specific optimization.

A.3 Software Stack

All experiments used:

- Chromium-based browser (stable release channel),
- default browser security settings (no extensions except GBV observation client),
- standard OS networking stack without kernel modification,
- GBV verifier and observation client built from the same versioned repository commit.

No privileged APIs, browser instrumentation frameworks, or trusted execution mechanisms were used.

A.4 Network and Scheduling Conditions

Experiments were conducted under typical residential broadband conditions.

To introduce realistic nondeterminism and avoid systematic bias from fixed execution order, experimental scheduling incorporated controlled randomization:

- inter-run delay randomized uniformly in $[2s, 15s]$,
- page reload points randomized between runs,
- verifier challenges randomized nonce values and surface ordering.

Such randomized execution mitigates measurement artifacts known to arise from deterministic benchmarking workflows [14].

CPU stress experiments introduced controlled background load using independent processes, while network jitter experiments applied bandwidth throttling and latency injection using standard traffic-shaping tools.

A.5 Surface Plans and Invariant Configuration

Surface plans $\mathcal{L}_k$ and invariant sets $\mathcal{C}_k$ are versioned configuration artifacts separate from protocol code.

Each configuration specifies:

- traversal procedures for each observation surface,
- extraction rules mapping artifacts to semantic facts,
- invariant constraints (agreement, consistency, presence, uniqueness),
- nonce embedding locations and freshness linkage requirements.

Evaluation sweeps vary only configuration parameters; verifier and client implementations remain unchanged across applications.

A.6 Execution Procedure

Each evaluation run follows the same deterministic procedure:

1. Verifier generates challenge $Q = \langle \eta, \Delta, \mathcal{L}, \text{policy} \rangle$.
2. Observation client executes traversal plan $\mathcal{L}$ and captures artifacts.
3. Artifacts are transmitted to the verifier.
4. Verifier performs semantic extraction, invariant evaluation, canonicalization, and commitment construction.
5. Structured logs and timing metrics are recorded.

Runs terminate with either an acceptance decision or an explicit failure reason code.

A.7 Adversarial Experiment Methodology

Adversarial implementations were developed under controlled conditions using identical tooling and workflow across attacker families.

For each attacker:

- implementation began from a clean honest client baseline,

- modifications were introduced incrementally following failures,
- success was defined as verifier acceptance under unchanged configuration,
- development time includes implementation, debugging, and validation required to achieve a functioning attack.

Attacker implementations were held fixed during sensitivity sweeps over k to isolate the effect of GBV configuration rather than attacker redesign.

All attacker implementations were developed by the same experimenter under identical constraints to reduce variability in effort measurement.

A.8 Measurement and Aggregation

Timing measurements are collected at the verifier to avoid client-side clock manipulation.

Latency statistics report:

- median runtime (robust central tendency),
- P95 tail latency (user-visible worst-case behavior),

consistent with standard large-scale systems performance reporting practices emphasizing tail behavior [7].

All reported metrics aggregate multiple independent runs as defined in Section 6. No runs were discarded post hoc; runs were included unless terminated by external interruption (e.g., manual process termination or system shutdown).

A.9 Replay and Verification

Because GBV verification is deterministic with respect to submitted artifacts, stored evidence bundles can be replayed offline, enabling independent validation consistent with prior work on deterministic verification and analysis [20].

Replay scripts reconstruct verification outcomes by re-running:

- semantic extraction,
- invariant evaluation,
- canonicalization, and
- commitment generation

against recorded artifact bundles.

A.10 Limitations of Reproducibility

Browser-based systems inherently exhibit environmental variance due to asynchronous execution, network timing, and rendering behavior. Accordingly, reproduced latency distributions may vary slightly across environments; however, acceptance decisions remain deterministic for identical artifact sets.

To preserve double-blind review, repository identifiers and build instructions are intentionally omitted from this submission. The full implementation and evaluation artifacts will be released as an open-source artifact upon publication, enabling independent reproduction of all reported results.

References

- [1] William Arbaugh, David Farber, and Jonathan Smith. A secure and reliable bootstrap architecture. In *IEEE Symposium on Security and Privacy*, 1997.
- [2] Algirdas Avizienis. The n-version approach to fault-tolerant software. *IEEE Transactions on Software Engineering*, (12):1491–1501, 1985.
- [3] Adam Barth, Collin Jackson, and John C. Mitchell. Securing frame communication in browsers. In *Proceedings of the 17th USENIX Security Symposium*, pages 17–30. USENIX Association, 2008.
- [4] Henk Birkholz, Dave Thaler, Michael Richardson, Ned Smith, and Weiming Pan. Remote attestation procedures (rats) architecture. RFC 9334, 2023.
- [5] Miguel Castro and Barbara Liskov. Practical byzantine fault tolerance. In *Proceedings of the 3rd Symposium on Operating Systems Design and Implementation (OSDI)*, 1999.
- [6] Victor Costan and Srinivas Devadas. Intel sgx explained. Technical Report 2016/086, IACR Cryptology ePrint Archive, 2016.
- [7] Jeffrey Dean and Luiz André Barroso. The tail at scale. *Communications of the ACM*, 56(2):74–80, 2013.
- [8] Mustafa E. Dincturk, Shauvik Roy Choudhary, Arie von Deursen, and Ali Mesbah. A model-based approach for crawling rich internet applications. *ACM Transactions on the Web (TWEB)*, 8(3):19:1–19:39, 2014.
- [9] Collin Jackson and Adam Barth. Beware of finer-grained origins. In *Web 2.0 Security and Privacy (W2SP)*, 2008.
- [10] Leslie Lamport, Robert Shostak, and Marshall Pease. The byzantine generals problem. *ACM Transactions on Programming Languages and Systems*, 4(3):382–401, 1982.
- [11] Ralph C. Merkle. Protocols for public key cryptosystems. In *IEEE Symposium on Security and Privacy*, 1980.
- [12] Ralph C. Merkle. A digital signature based on a conventional encryption function. In *Advances in Cryptology – CRYPTO ’87*, volume 293 of *Lecture Notes in Computer Science*, pages 369–378. Springer, 1988.
- [13] Ali Mesbah and Mukul R. Prasad. Automated cross-browser compatibility testing. In *International Conference on Software Engineering (ICSE)*, pages 561–570, 2011.
- [14] Todd Mytkowicz, Amer Diwan, Matthias Hauswirth, and Peter F. Sweeney. Producing wrong data without doing anything obviously wrong. In *Proceedings of the 14th International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS)*, pages 265–276. ACM, 2009.
- [15] Bryan Parno, Jonathan M. McCune, and Adrian Perrig. Bootstrapping trust in commodity computers. In *IEEE Symposium on Security and Privacy (S&P)*, pages 414–429, 2010.
- [16] Charles Reis, Steven D. Gribble, et al. Isolating web programs in modern browser architectures. In *Proceedings of the 4th ACM European Conference on Computer Systems (EuroSys)*, 2009.
- [17] Eric Rescorla. The transport layer security (tls) protocol version 1.3. RFC 8446, 2018.
- [18] Reiner Sailer, Xiaolan Zhang, Trent Jaeger, and Leendert van Doorn. Design and implementation of a tcb-based integrity measurement architecture. In *USENIX Security Symposium*, 2004.
- [19] Koushik Sen, Swaroop Kalasapur, Thomas Brutch, and Simon Gibbs. Jalangi: A selective record-replay and dynamic analysis framework for javascript. In *Proceedings of the 2013 9th Joint Meeting on Foundations of Software Engineering (ESEC/FSE)*, 2013.
- [20] David A. Wheeler. Countering trusting trust through diverse double-compiling. In *Annual Computer Security Applications Conference (ACSAC)*, pages 28–40, 2005.